

B.Sc. (Data Science)

Unit No.1: Introduction to Problem Solving

Unit No.1: Introduction to Problem Solving

1.1 What is problem solving?

1.2 Problem solving steps.

1.3 Algorithms - definition, characteristics, examples, advantages and limitations.

1.4 Flowcharts - definition, notations, examples, advantages and limitations, Comparison with algorithms.

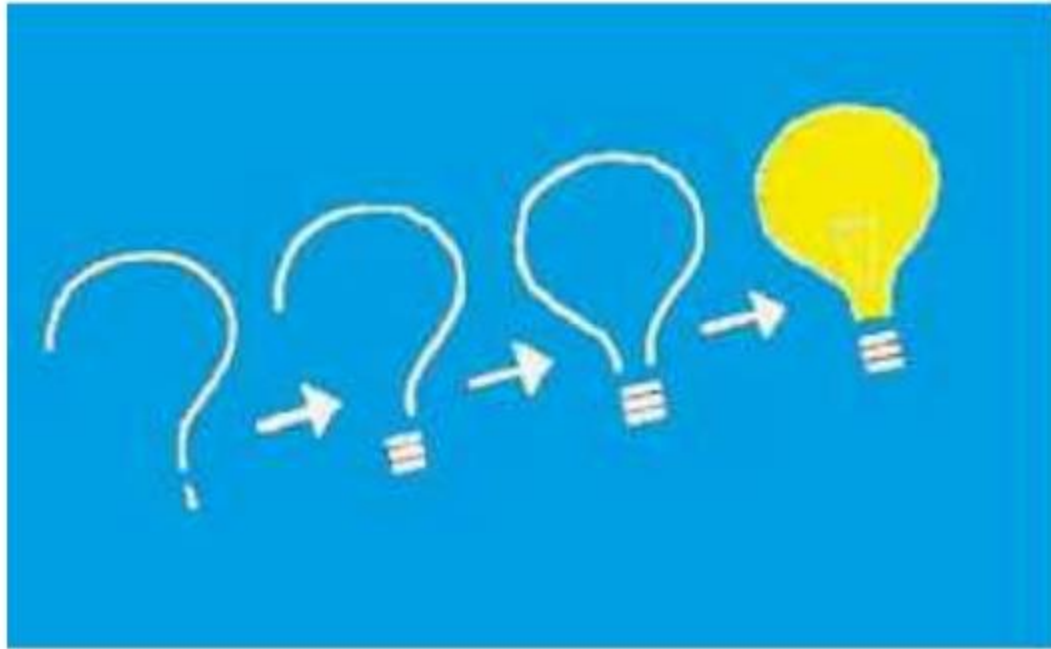
1.5 Pseudo codes - notations, examples, advantages and limitations.

1.6 Introduction to Programming

1.7 Programming Languages as tools, programming paradigms, types of languages

1.8 Converting pseudo-code to programs.

What is problem solving?



What is Problem and What is Problem Solving?

- Problem is a situation regarded as unwelcome and which needs to be dealt with and overcome.
- Problem solving is the process of transforming the description of a problem into solution of that problem by using our knowledge of the problem domain.
- It refers to analyzing a problem with the intention of deriving a solution for the problem.

Problem solving steps

- ☐ Understand the problem
- ☐ Generate alternative solutions
- ☐ Evaluate and select an alternative
- ☐ Implement and follow up on the solution

Example of Problem Solving

- How to attend Online Lectures Today?

Problem Solving Wheel



Before Writing a Program

- Before a programmer actually writes a program the programmer must plan the program.
- An algorithm helps to outline the Program.
- In this we design/ plan the Program based on particular procedure/steps.
- This steps need to be followed in a sequence to get the result.

Definition of Algorithm and Characteristics

Algorithm:- It is a step by step procedure of solving a particular problem.

It is a finite set of instruction in a specific sequence which if followed accomplishes a particular task.

Characteristics of an Algorithm:-

- **Input:** - Every algorithm should have zero or more input.
- **Output:** - Algorithm should produce at least one output.
- **Finiteness:** - An algorithm should terminate after finite number of steps.
- **Definiteness:** - Each step of the algorithm should be clear and unambiguous.
- **Effectiveness:** - An algorithm should be effective. Each step should be referred as principle and should be executed in finite time.

Advantages of Algorithm

- It is a general purpose tool which is language and hardware independent.
- It is easy to read write and understand.
- It uses a definite procedure.
- It is easy to debug or Identification of errors is easier.
- It breaks problem into smaller pieces hence it is easy to convert it to a program.
- It is not dependent on any programming language so easy to understand without programming knowledge.

Disadvantages of Algorithm

- Time consuming process.
- Very big tasks are difficult to write.
- Difficult to show branching and repetitive task.
- For big task an algorithm can become lengthy and complicated.

Simple Algorithm for making a Cup of Tea

- Step 1:- Start
- Step 2:- Put the teabag in a cup.
- Step 3:- Fill the kettle with water.
- Step 4:- Boil the water in the kettle.
- Step 5:- Pour some of the boiled water into the cup.
- Step 6:- Add milk to the cup.
- Step 7:- Add sugar to the cup.
- Step 8:- Stir the tea.
- Step 9:- Drink the tea
- Step 10:- Stop

Algorithm for using Gmail Account

- **Step 1:- Start**
- **Step 1:- Open the Gmail homepage.** Use your web browser and navigate to the Gmail home page and click sign in.
- **Step 2:- Enter your email address.** Enter the valid email address you have used to create your account.
- **Step 3:- Enter your password.** You will need to enter your password that you created in order to log in. If you forgot your password, click the “Forgot password” link below the sign-in field. If username and Password matches correctly then you are successfully logged in into your account.
- **Step 4:- Stop**

Flowcharts

- It is a pictorial representation of algorithm.

Advantages of Flowchart:-

1. Flowcharts are better way of communicating the logic of a system to all concerned or involved.
2. With the help of flowchart, problem can be analyzed in more effective way therefore reducing cost and wastage of time.
3. Program **flowcharts** serve as a good program documentation, which is needed for various purposes, making things more efficient.
4. The flowchart helps in debugging process.
5. Branching and Looping can be easily described.
6. It is easy to write program from flowchart.

Disadvantages of flowchart

- It is a time consuming process.
- Sometimes, the program logic is quite complicated. In that case, flowchart becomes complex and clumsy.
- If alterations are required the flowchart may require re-drawing completely. This will usually waste valuable time.

Basic Notations used in Flowcharts

Symbol	Name	Use
	Terminal	To indicate the entry and exit points of the flowchart.
	Input/ Output	It indicates input or output task.
	Flow lines	Acts as connectors of all other symbols.
	Process	To show processing steps like calculations or an action to be performed.
	Decision	Decision making step. Generally, there are two outward connectors from this symbol. The path is chosen based on the decision of the logical computation
	Connectors	If a flowchart does not fit in the available space, and needs to be broken at a point. An alphanumeric symbol within a circle can be used as the breaking and continuing symbol for that flowchart.

Algorithm for Addition of two numbers

STEP 1 : Start

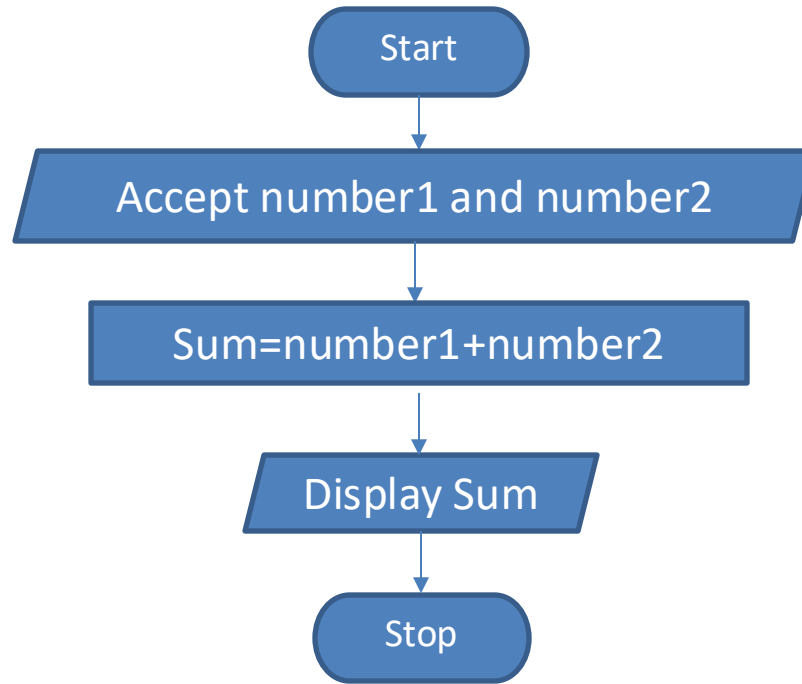
STEP 2 : Accept Number1 and Number2

STEP 3 : Add these two numbers $\text{Sum} = \text{Number1} + \text{Number2}$

STEP 4 : Display Sum

STEP 5 : Stop

Flowchart for Addition of two numbers



Algorithm for finding Area of Circle

STEP 1 : START

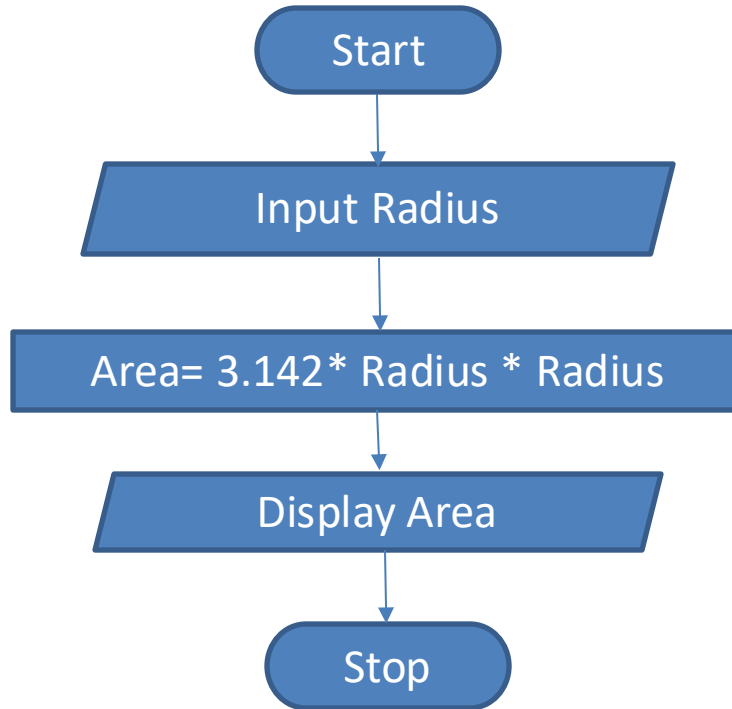
STEP 2 : ACCEPT Radius value from user.

STEP 3 : Calculate $\text{Area} = 3.142 * \text{Radius} * \text{Radius}$

STEP 4 : Display Area

STEP 5 : STOP

Flowchart for Area of Circle



Algorithm for finding Area of Rectangle

STEP 1 : START

STEP 2 : ACCEPT THE LENGTH L OF RECTANGLE

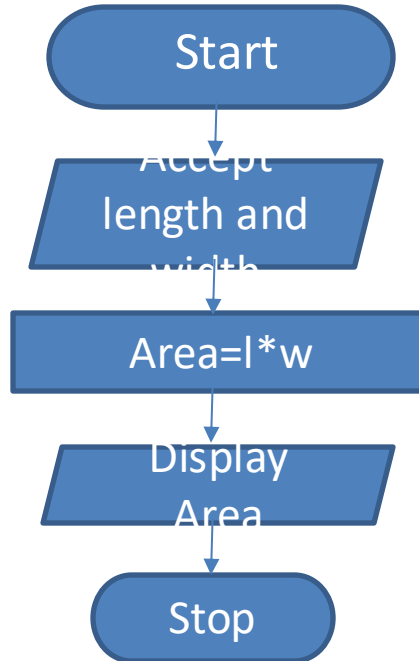
STEP 3 : ACCEPT THE WIDTH W OF RECTANGLE

STEP 4 : Area = $L * W$

STEP 5 : DISPLAY Area

STEP 6 : STOP

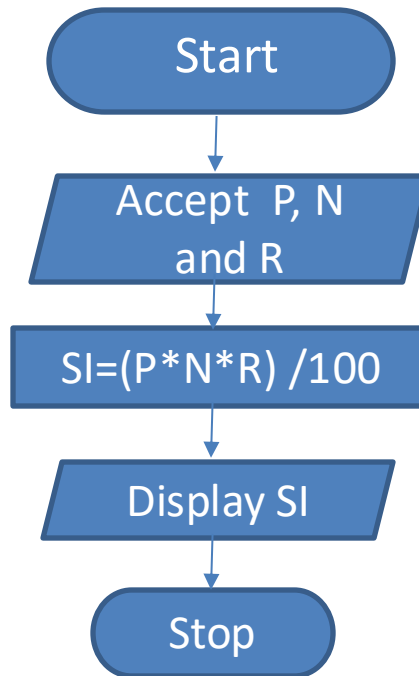
Flowchart for Area of Rectangle



Algorithm for finding Simple Interest

- STEP 1 : START
- STEP 2 : ACCEPT Values of P, N and R
- STEP 3 : $SI = (P * N * R) / 100$
- STEP 4 : DISPLAY SI
- STEP 6 : STOP

Flowchart for finding Simple Interest



Algorithm to convert temperature from Celcius to Farheneit

STEP 1 : START

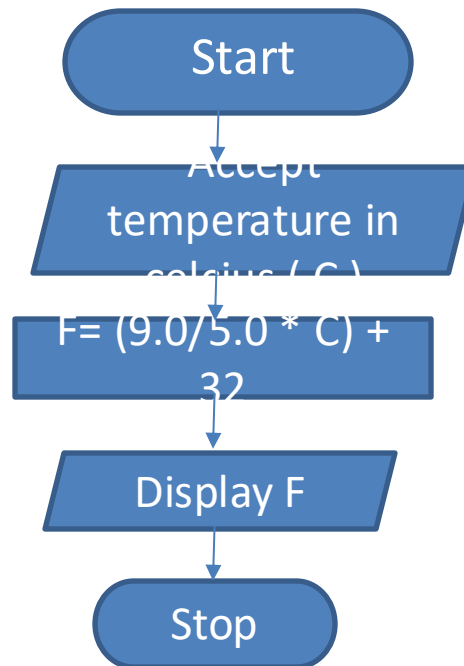
STEP 2 : ACCEPT Temperature in Celcius

STEP 3 : $F = (9.0/5.0 * C) + 32$

STEP 4 : Display temperature in F

STEP 5: STOP

Flowchart for converting temperature



Comparison of Algorithm and Flowchart

Algorithm	Flowchart
It includes stepwise procedure to solve a problem	It includes a diagrammatic representation of algorithm.
It uses simple English language	It uses symbols
Easy to debug	Difficult to debug
Hard to understand	Easy to interpret

Decision Control Structure

- In sequence control structure all the statements are executed sequentially.
ie: in the same order as they appear in the program.
- But many times we need a block of statements to be executed in one situation and other block of statements to be executed in other situation. This is Decision control structure.

- **CONDITIONAL**: In a conditional statement you make a test. The result of the test is a Boolean - either True or False. If the result of the test is True you take a certain course of action and if the result of the test is False you take another course of action.

IF condition THEN

sequence 1

ELSE

sequence 2

Determine and Output Whether Number N is Even or Odd

- **Algorithm:**

Step 1: Read number N

Step 2: Set remainder as $N \text{ modulo } 2$,

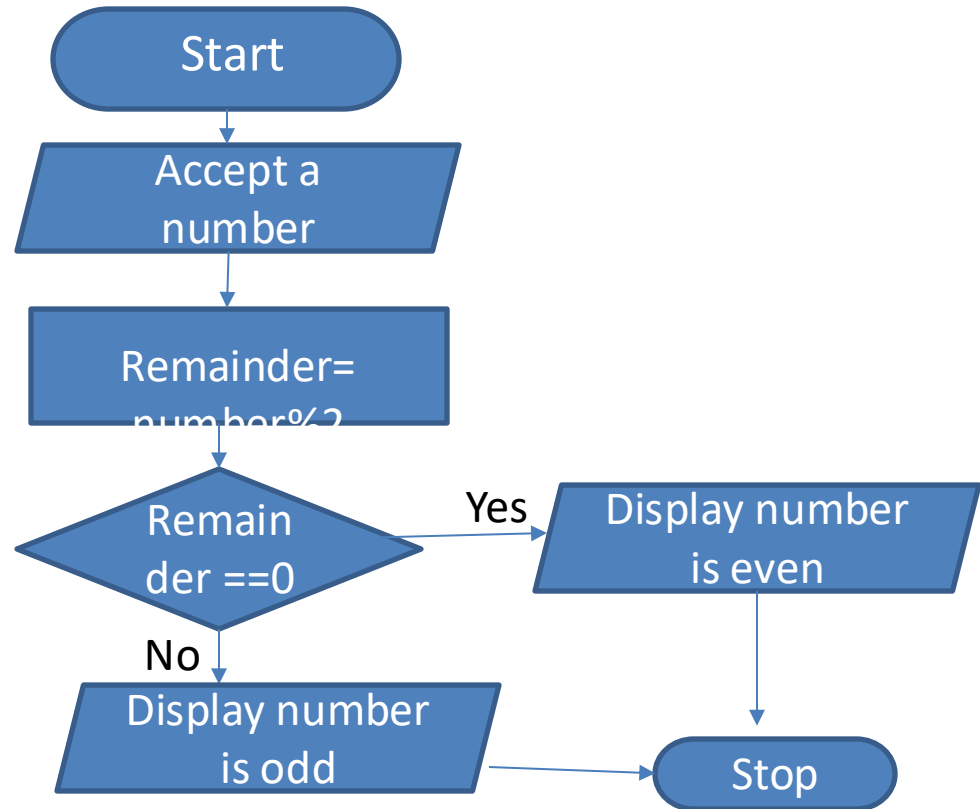
Step 3: If the remainder is equal to 0 then goto step 4 else goto step 5

Step 4: Display number is even

Step 5:- Display number is odd

Step 6:- Stop

To check if number is even or odd



Determine Whether a Temperature is Below or Above the Freezing Point

- **Algorithm:**

Step 1: Accept temperature

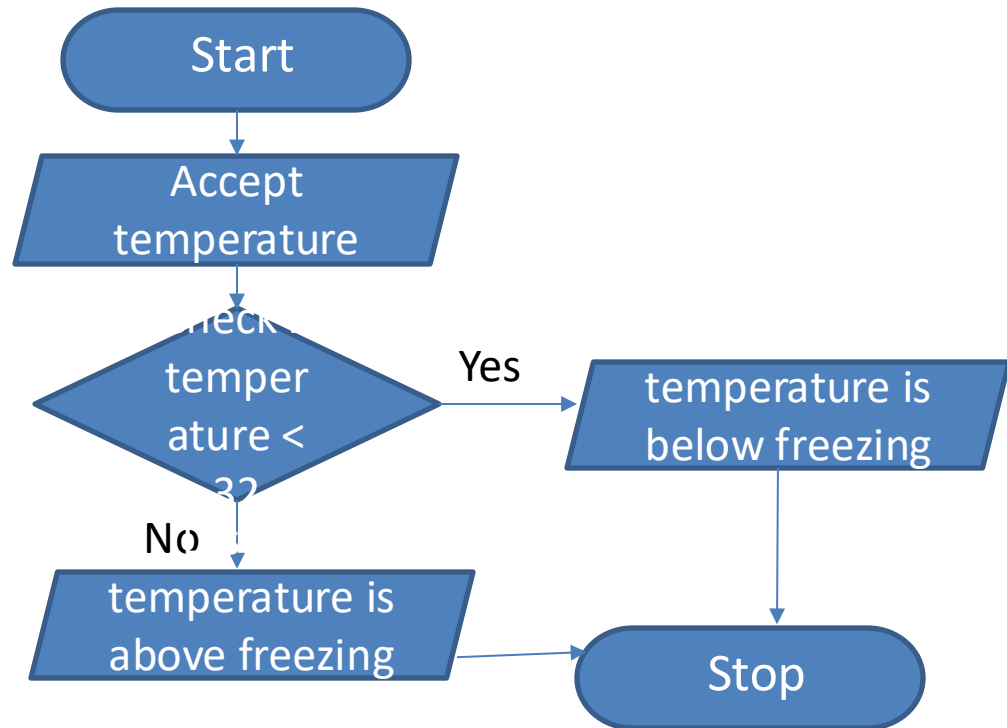
Step 2: Check If temperature is less than 32, if true then goto step 3 else goto step 4

Step 3: print "Temperature is below freezing point."

Step 4:- Print "Temperature is above freezing point".

Step 5:- Stop

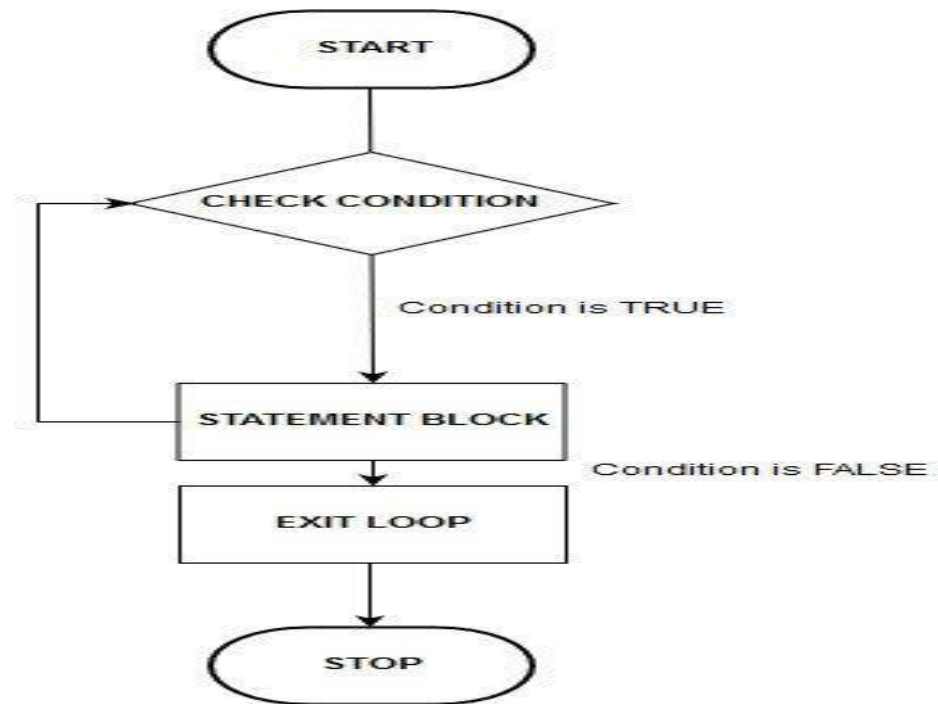
Flowchart



Iteration

- **Iteration:** Algorithms often use repetition to execute steps a certain number of times or until a certain condition is met.
- The For/While Loop is a loop where the program tells the compiler to run a specific code FOR a specified number of times.
- This loop allows using three statements, first is the counter initialization, next is

C For Loop Flowchart



Algorithm to print sum of first 10 numbers

Step 1. Start

Step 2. Initialize $i = 1$, $SUM = 0$

Step 3. if ($i > 10$) goto step 7 else goto step 4

Step 4. $SUM = SUM + i$

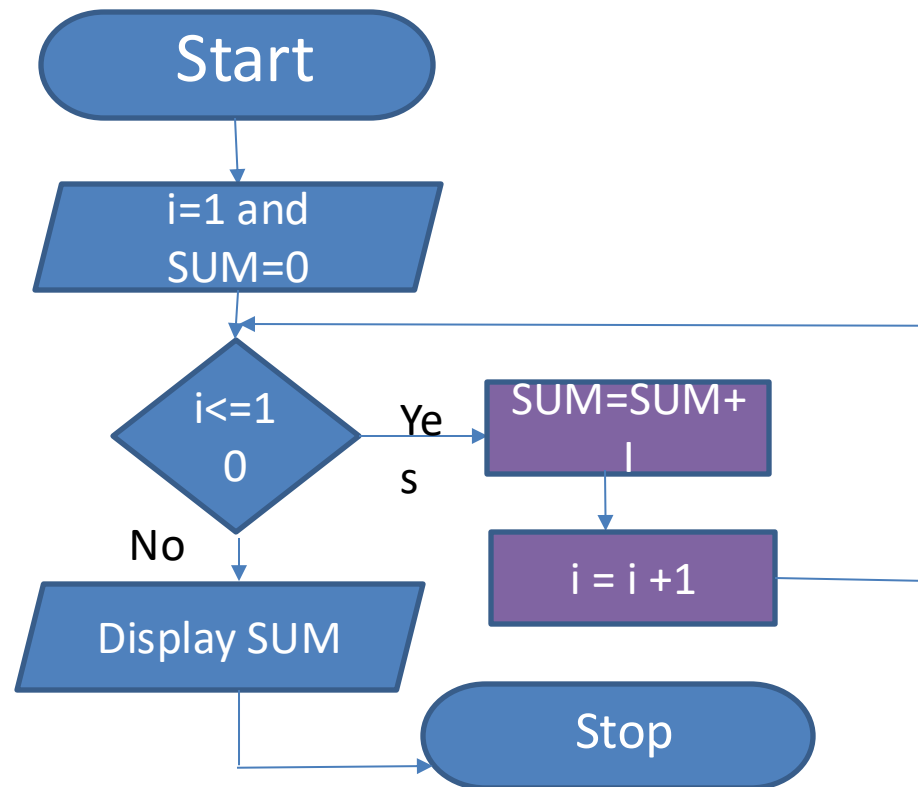
Step 5. $i = i + 1$

Step 6. go to step 3

Step 7. Display the value of SUM

Step 8. Stop

Flowchart to print sum of first 10 numbers



Algorithm to print sum of digit of a number

Step 1:- Start

Step 2:- Read a number

Step 3:- $\text{sum} = 0$

Step 4:- if $n > 0$ then goto next step else goto step 9

Step 5:- $\text{last} = n \bmod 10$

Step 6:- $\text{sum} = \text{sum} + \text{last}$

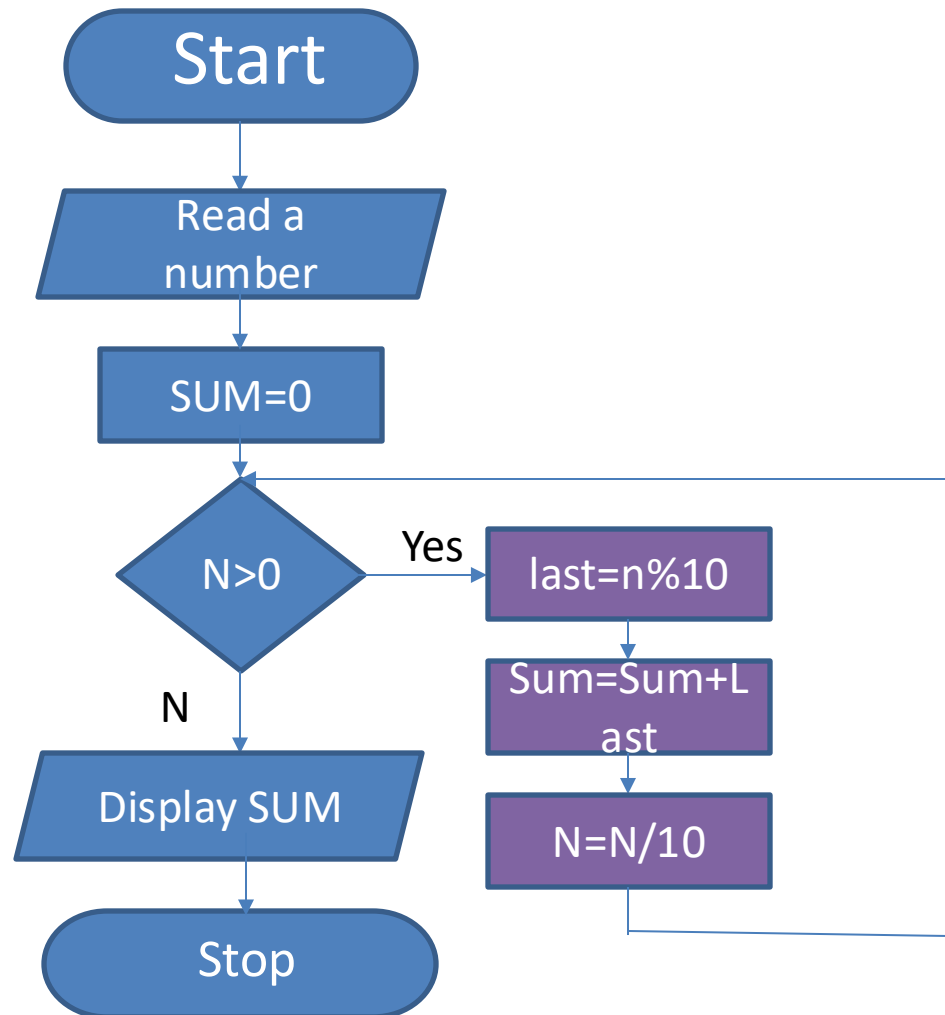
Step 7:- $n = n / 10$

Step 8:- goto step 4

Step 9:- Display sum

Step 10:- Stop

Flowchart to print sum of digit



Algorithm to reverse a number

Step 1:- Start

Step 2:- Read a number n

Step 3:- $rev = 0$

Step 4:- if $n < 0$ then goto step 8

Step 5:- $r = n \% 10$

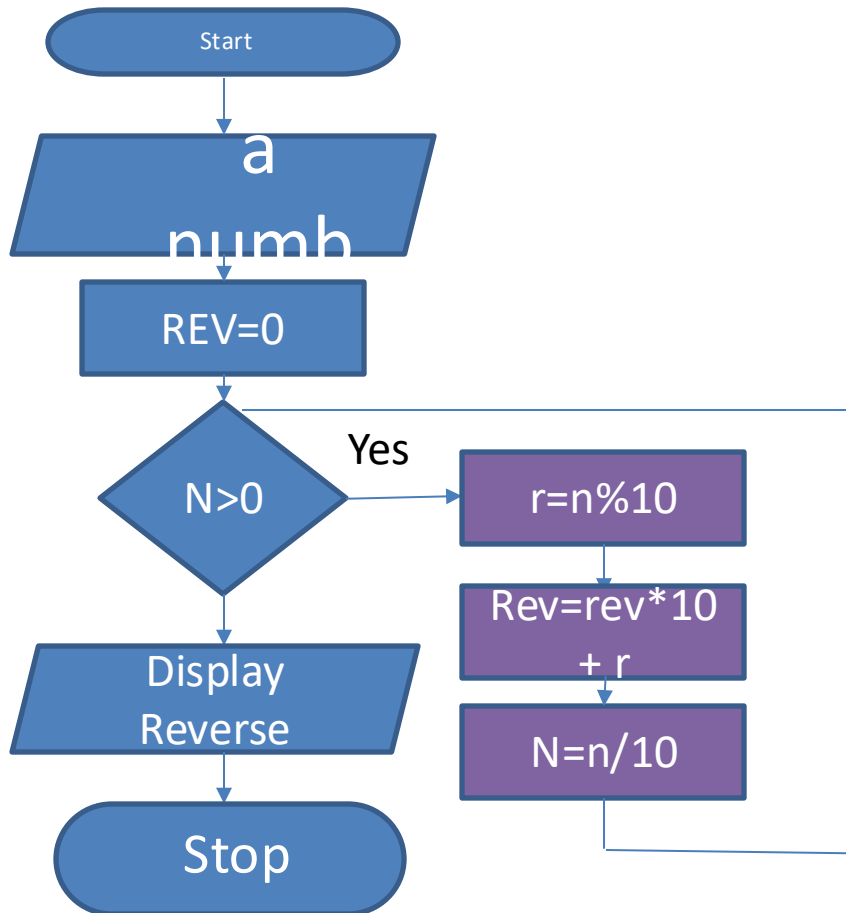
Step 6:- $rev = rev * 10 + r$

Step 7:- $n = n / 10$

Step 8: Display rev

Step 9:= Stop

Flowchart to print reverse a number



Check if number is Armstrong or not

Step 1: Start

Step 2: Read num from User

Step 3: Initialize sum=0 and temp=num

Step 4: Repeat Until num>=0

Step 5: sum=sum + cube of last digit i.e

$[(\text{num}\%10)*(\text{num}\%10)*(\text{num}\%10)]$

Step 6: num=num/10

Check if number is Armstrong or not

Step 8: IF sum==temp

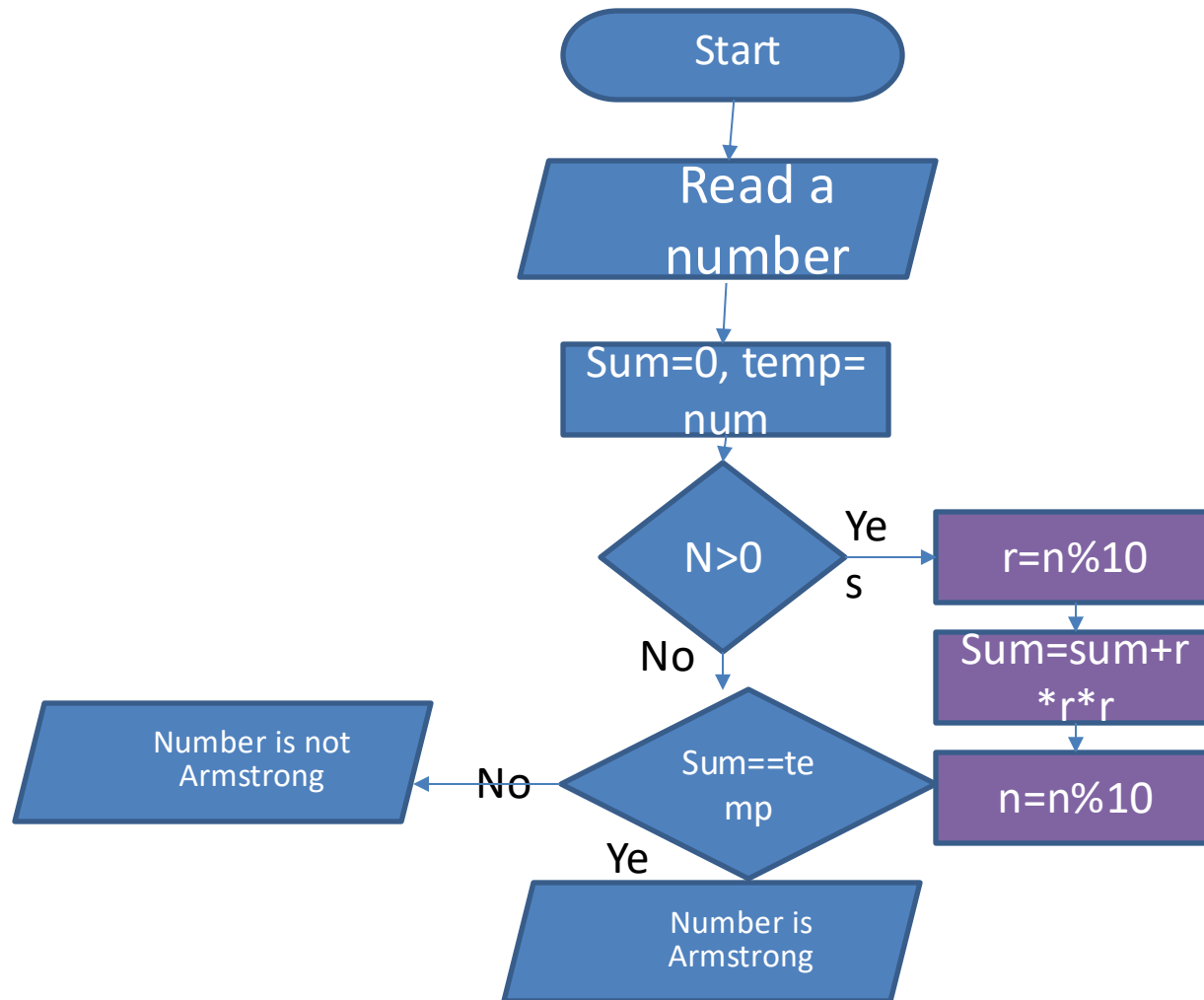
Print "Armstrong Number"

ELSE

Print "Not Armstrong Number"

Step 9: Stop

Flowchart



Algorithm and Flowchart

- Write an Algorithm and Flowchart to accept three dimensions length (l), breadth(b) and height(h) of a cuboid and print surface area
$$\text{surface area} = 2 * (l * b + l * h + b * h)$$
- Write an Algorithm and Flowchart to Accept dimensions of a cylinder and print the surface area and volume. ($\text{surface area} = 2\pi r^2 + 2\pi r h$, $\text{volume} = \pi r^2 h$)

Algorithm and Flowchart

- **Write an Algorithm and Flowchart to accept initial velocity (u) , acceleration (a), and time (t). Print the final velocity (v). (Use formula $v = u + at$)**

Algorithm and Flowchart

- **Write an Algorithm and Flowchart which accepts annual basic salary of an employee and calculates and displays the Income tax as per the following rules.**

Basic: $< 1,50,000$ Tax = 0

1,50,000 to 3,00,000 Tax = 20%

$> 3,00,000$ Tax = 30%

Algorithm and Flowchart

- **Write a 'C' program to accept marks of 5 subjects of a student and calculate the grade using rules:**
 - > 90 grade = O**
 - between 75 and 90 grade = A**
 - between 60 and 75 grade = B**
 - between 40 and 60 grade = C**
 - < 40 grade = F**

Algorithm and Flowchart

- Write a 'C' program to read the age of a candidate and determine whether the candidate is eligible for casting his/her own vote.
- Write a 'C' program to accept an integer and check if it is divisible by 3 and 5.
- Write a 'C' program to check whether the given year is leap year or not.
- Write a 'C' program to accept a character and

Home Assignments

- Algorithm & Flowchart to find Area and Perimeter of Circle
- Algorithm & Flowchart to find Area and Perimeter of Square

Comparison of Algorithm and Flowchart

Algorithm	Flowchart
It includes stepwise procedure to solve a problem	It includes a diagrammatic representation of algorithm.
It uses simple English language	It uses symbols
Easy to debug	Difficult to debug
Hard to understand	Easy to interpret

Pseudo Codes:

-

- It is a term which is often used in programming and algorithm based fields.
- It is a methodology that allows the programmer to represent the implementation of an algorithm.
- It's simply an implementation of an algorithm in the form of annotations and

Advantages of Pseudocode

1. Improves the readability of any approach. It's one of the best approaches to start implementation of an algorithm.
2. Acts as a bridge between the program and the algorithm or flowchart.
3. The main goal of a pseudo code is to

Disadvantages of Pseudocode

- While pseudocode is easy to read, it does not provide a good map for the programmer as a flowchart does.
- It does not include the full logic of the proposed code.

Pseudocode for Addition of two numbers

Input number1

Input number2

Calculate $\text{sum} = \text{number1} + \text{number2}$

Display sum

Pseudocode for finding maximum of two numbers

Input number1

Input number2

 If number1>number 2 then display
 number1 is greater than number 2

 Else Display number2 is greater than
 number 1

Pseudo code for sum of digits of a number

Input a Number

Initialize Sum to zero

While Number is not zero

 Get Remainder by Number Mod
10

 Add Remainder to Sum

 Divide Number by 10

Print sum

Pseudo code for Reverse of a number

Input a Number

Initialize rev to zero

While Number is not zero

 Get Remainder by Number Mod 10 ($r = n \% 10$)

 Add Remainder to rev * 10
($rev = rev * 10 + r$)

 Divide Number by 10 ($n = n / 10$)

Pseudo code for Sum of first 10 numbers

Input a Number

Initialize i to 1 and sum to zero

While i is not equal to 10

 Add sum to i and store it in sum

 Add i with 1 and store it in i

Print sum

Pseudocode to check if
number is Armstrong or not

READ n

temp=n

sum=0

WHILE n>=0

 sum=sum+(n%10)*(n%10)*(n%10)

 n=n/10

ENDWHILE

Pseudocode

IF sum==temp

 WRITE "NUMBER IS AN ARMSTRONG
NUMBER"

ELSE

 WRITE "NUMBER IS NOT AN ARMSTRONG
NUMBER"

Program: - A program is a collection of instruction that performs a specific task.

Programming Paradigm: - It is a style or way of programming. Some common paradigms are Imperative, Declarative, Structured, Procedural, Functional, Object oriented, Event driven, etc.

Language: - The method of human communication, either spoken or written, consisting of the use of words in a structured and conventional way

Types of languages:

There are basically two types of computer programming languages given below:

Low level language

High level language

Low Level Languages:– The programming languages that are very close to machine code (0s and 1s) are called low-level programming languages.

The program instructions written in these languages are in binary form.

The examples of low-level languages are:

Machine language

Assembly language

Machine Language

- The instructions in binary form, which can be directly understood by the computer without translating them, is called a machine language or machine code.
- Machine language is also known as first generation of programming language. Machine language is the fundamental language of the computer.
- The program instructions in this language is in the binary form (that is 0's and 1's).
- This language is different for different computers.
- It is not easy to learn the machine language.

Advantage and Disadvantages of Machine Language

- The only advantage of machine language is that the program of machine language runs very fast because no translation program is required for the CPU.

Disadvantage of Machine Language

- Machine language differs from one computer to another.
- Checking machine instructions to locate errors is very difficult and time consuming.
- It is difficult to program in machine language.
- A machine language programming must be knowledgeable about the hardware structure of the computer.

Assembly Language

- It is another low-level programming language because the program instructions written in this language are close to machine language.
- Assembly language is also known as second generation of programming language or Mnemonic Language.
- With assembly language, a programmer writes instructions using symbolic instruction code instead of binary codes.
- Symbolic codes are meaningful abbreviations.
- The set of program instructions written in assembly language are also called as mnemonic code.
- Assembly language provides facilities for controlling the hardware.

Advantages and Disadvantages

It is much easier to understand compared to machine language.

- It is easy to detect errors.
- Assembly language are easier to read, write and understand.

Disadvantage of Assembly Languages

- It requires a translator.
- Assembly languages are machine dependent, an assembly language programmer must have a

High Level Language

- High Level Languages The programming languages that are close to human languages (example like English languages) are called the high-level languages.
- The examples of high-level languages are:
Fortran, COBOL, Basic, Pascal, C, C++, Java

High Level Language

- The program instructions are written using English words, for example print, input etc. But each high level language has its own rule and grammar for writing program instructions.
- These rules are called syntax of the language.

Advantages of High Level Languages

- **Easy to learn** - the high level languages are very easy to learn than low level languages. The statements written for the program are similar to English-like statements.
- **Easy to understand** - the program written in high level language by one programmer can easily be understood by another because the program instructions are similar to the English language.
- **Easy to write program** - in high level language, a new program can easily be written in a very short time. The larger and complicated software can be developed in few days or months.
- **Easy to detect and remove errors** - the errors in a program can be easily detected and removed. mostly the errors are occurred during the compilation of new program.
- **Built-in library functions** - Each high level language provides a large number of built-in functions or procedures that can be used to perform specific task during designing of new programs. In this way, a large amount of time of programmer is saved.
- **Machine Independence** - program written in high level language is machine independent. It means that a program written in one type of computer can be executed on another type of computer.

Limitation of High Level Language

Low efficiency - a program written in high level languages has lower efficiency than one written in a machine/assembly language to do the same job. That is, program written in high level languages result in multiple machine language instruction that may not be optimize, taking more time to execute and requiring more memory space.

Procedural languages

- Procedural languages are also known as third generation languages (3GLs).
- In a procedural language, a program is designed using procedures.
- A procedure is a sequence of instructions having a unique name.
- The instructions of the procedure are executed with the reference of its name.
- In procedural programming languages, the program instructions are written in a sequence or in a specific order in which they must be executed to solve a specific problem.

Translator: - is a general term which refers to conversion of code from one form to another. It could refer to a compiler, assembler or Interpreter.

Compiler: - A compiler is a program that translates source language to target language.

Interpreter: - It is a program that transforms

Difference between Compiler and Interpreter

Compiler	Interpreter
It scans the entire program and translates it as a whole into machine code.	It translates the program statement by statement
It takes large amount of time to analyze the source code but the overall execution time is comparatively faster	It takes less amount of time to analyze the source code but the overall execution time is slower.
Generates intermediate object code which further requires linking, hence requires more memory.	No intermediate object code is generated, hence are memory efficient.
Programming language like C, C++ use compilers.	Programming language like Python, Ruby use interpreters.
It generates the error message only after scanning the whole program. Hence debugging is comparatively hard.	Continues translating the program until the first error is met, in which case it stops. Hence debugging is easy.

Types of Errors

Syntax errors: – A syntax error is an error in the syntax or coding entered by programmer.

Compiler is used to find syntax errors. For

Example: A missing semicolon at the end of line is a syntax error, missing Brackets, Printing values of variables without declaring it.

Semantic Errors: – Such type of errors occur due to improper use of program statement or

Good programming practices

- **Naming conventions** Use of proper naming conventions is a well-known best practice.
- Is a very common issue where developers use variables like X1, Y1 and forget to replace them with meaningful ones, causing confusion and making the code less readable.
- Keep the code simple The code should always be simple

Good programming practices

- So, always keep the code as simple as possible.
- Limit line length Long lines are hard to read.
- It is a good practice to avoid writing horizontally long lines of code
- **Documentation:** - It is necessary to properly document your program so that it will be easy

Indentation is one of the most important features in any programming domain. In computer programming, an indent style is a convention governing the indenting of blocks of code to convey program structure. Indentation helps to convey a better structure of a program to the readers. Proper code indentation will make code.

Easy to read

MCQ

1. The symbol denotes _____

a) I/O

b) Flow

c) Terminal

d) Decision

Q.2

2. Which of the following is not an advantage of a flowchart?

- a) Better communication
- b) Efficient coding
- c) Systematic testing
- d) Improper documentation**

Q.3

3. Method which uses a list of well defined instructions to complete a task from the given initial state is called as _____

- A. Program
- B. Flowchart
- C. Algorithm**
- D. A and b

Q.4

4. Which of the following logic is used to produce loops in program logic when one or more instructions are executed

- a. Iteration Logic**
- b. Sequence logic
- c. Selection logic
- d. Decision logic

Q.5.

5. Algorithm is a

a) Step by step execution

b) Object file

c) Executable file

d) Source file

Q.6

6.The following pseudo code is an example of :- Get number

- Get another number
- Multiply numbers
- Print result

A)Sequence

B) Decision

C) Loops

D) Nested

Q.7.

7. _____ scans the entire program and converts it into object code.

a.) **Compiler**

b.) Interpreter

c.) Algorithm

d.) Pseudocode

Q.8

8. C language is an example of _____

- a) Low level language
- b) Assembly language
- c) High Level Language**
- d) Binary Language

Q.9.

9. _____ language program runs fast as no translation is required.

- a) **Machine language**
- b) High Level language
- c) Assembly language
- d) Middle level language

Q.10

10. ____ language programs are machine dependent.

A) Assembly language

B) High level language

C) Middle level language

D) None of the above

Thank you